

Article

Comparative Read Performance Analysis of PostgreSQL and MongoDB in E-Commerce: An Empirical Study of Filtering and Analytical Queries

Jovita Urnikienė ^{1,*} , Vaida Steponavičienė ¹  and Svetoslav Atanasov ^{2,*} ¹ Department of Computer Science and Electrical Engineering, Faculty of Business and Technologies, Šiauliai Valstybinė Kolegija, 76241 Šiauliai, Lithuania; v.steponaviciene@svako.lt² Department of Electrical Engineering, Automation, Computer Systems and Communications, Trakia University, 8600 Yambol, Bulgaria

* Correspondence: j.urnikiene@svako.lt (J.U.); svetoslav.atanasov@trakia-uni.bg (S.A.)

Abstract

This paper presents a comparative analysis of read performance for PostgreSQL and MongoDB in e-commerce scenarios, using identical datasets in a resource-constrained single-host environment. The results demonstrate that PostgreSQL executes complex analytical queries 1.6–15.1 times faster, depending on the query type and data volume. The study employed synthetic data generation with the Faker library across three stages, processing up to 300,000 products and executing each of 6 query types 15 times. Both filtering and analytical queries were tested on non-indexed data in a controlled localhost environment with PostgreSQL 17.5 and MongoDB 7.0.14, using default configurations. PostgreSQL showed 65–80% shorter execution times for multi-criteria queries, while MongoDB required approximately 33% less disk space. These findings suggest that normalized relational schemas are advantageous for transactional e-commerce systems where analytical queries dominate the workload. The results are directly applicable to small and medium e-commerce developers operating in budget-constrained, single-host deployment environments when choosing between relational and document-oriented databases for structured transactional data with read-heavy analytical workloads. A minimal indexed validation confirms that the baseline trends remain consistent under a simple indexing configuration. Future work will examine broader indexing strategies, write-intensive workloads, and distributed deployment scenarios.

Keywords: relational databases; NoSQL databases; query execution performance; performance benchmarking; database selection



Academic Editor: Alberto Abelló

Received: 29 December 2025

Revised: 14 February 2026

Accepted: 17 February 2026

Published: 19 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

The field of e-commerce is one of the most data-driven business domains, facing not only increasing data flows but also a growing number of virtual customers and queries. This creates a practical need for systems that store data efficiently in databases. When deciding which type of database to use—relational (SQL-based) or document-oriented (NoSQL)—it is essential to evaluate performance differences under realistic conditions.

Although both relational and document databases are widely used, their performance differences in real business situations—especially in e-commerce—have not yet been fully empirically assessed using identical datasets and comparable scenarios. Many existing comparisons are theoretical or focused on large-scale systems [1–4], which often do not

reflect the needs of small- or medium-sized businesses. For this reason, system developers implementing e-commerce solutions face challenges when selecting the optimal database architecture to ensure fast data processing and system stability.

Matcha S. et al. [4] emphasize that selecting an appropriate database system affects system performance, management, and data growth. It is emphasized that the choice of database should be based on specific business needs and data characteristics rather than general standards. The publication presents a general comparison of SQL and NoSQL databases, examining their performance, applicability, and data management capabilities. However, the study does not provide detailed methodological information, such as the type and volume of data used or the queries and operations tested, which limits its applicability to specific e-commerce contexts.

Similar observations at the system level are presented in a comprehensive literature review by Taipalus, T. [1], which systematically compares performance across database management systems. The review notes that many existing studies do not reflect real-world use cases and emphasizes that database selection should not be based solely on performance indicators, as other aspects—such as the data model, environmental parameters, and business requirements—are equally important.

Khan W. et al. [2] provide a systematic literature review highlighting essential architectural and functional differences between SQL and NoSQL databases. The authors emphasize that relational systems rely on a structured data model and ensure consistency based on ACID (Atomicity, Consistency, Isolation, Durability) principles. In contrast, document-oriented databases focus on flexibility, do not require a strictly predefined data structure, support easier scaling as data volumes grow, and enable the storage and processing of diverse data types. The review concludes that there is still a lack of empirical comparative studies that apply different SQL and NoSQL database systems across various business contexts.

Hassan M. [3] discusses the importance of database system selection in the context of big data, agreeing with previous authors that the choice is influenced not only by technological characteristics but also by data volume, structure, frequency, and complexity of operations. Nevertheless, as in the other reviewed sources, this is primarily a theoretical overview without direct experimental validation.

Unlike the previously reviewed articles [1–4], Cheng Y., Zhou K., and Wang J. [5] conduct an experimental comparison of PostgreSQL and MongoDB, analyzing database performance when storing unstructured GeoTIFF data. Their findings show that both databases demonstrate different strengths depending on the scenario: MongoDB is efficient for rapid storage of unstructured data, whereas PostgreSQL performs better for precise data structures and smaller data volumes.

An additional experimental example is provided by Makris A. et al. [6], who compare the performance of MongoDB and PostgreSQL when executing spatio-temporal queries using real AIS (Automatic Identification System) maritime traffic data. The dataset used in the study comprised 146 million records spanning three months in the Mediterranean Sea. Both databases were built using the same data source, and identical queries were executed to ensure a fair comparison. The results showed that PostgreSQL generally delivered better response times, particularly for complex queries, while MongoDB demonstrated higher efficiency for certain geographic operations. However, these findings are not directly applicable to e-commerce, which relies on structured data, specific business scenarios, and diverse query types.

Considering the theoretical and experimental sources discussed, it is clear that choosing a database system in a business environment cannot be based solely on performance metrics or general technological guidelines. In the context of e-commerce, not only the

speed of data processing but also data structure, security, access control, and system flexibility become crucial as data volumes grow.

This study is positioned within the framework of experimental database systems research and benchmarking methodology. Following the workload characterization principles established by Cooper et al. [7] in the Yahoo! Cloud Serving Benchmark (YCSB), we categorize our test queries into OLTP-style filtering operations and OLAP-style analytical queries. Based on database architecture theory regarding query optimization in relational versus document-oriented systems [1,2,6], we hypothesize that: (H1) PostgreSQL is expected to demonstrate superior performance for join-intensive analytical queries; (H2) MongoDB is expected to demonstrate comparable performance for simple filtering operations.

Building on this foundation, the present study seeks to experimentally assess the performance of relational and document-oriented databases in typical e-commerce scenarios by modelling data filtering operations and complex aggregating queries. The primary contribution of this research is a controlled empirical comparison using identical datasets, standardized query scenarios, and transparent methodology that addresses the methodological gaps identified in the previous literature.

2. Background and Theoretical Framework

The comparison of relational and NoSQL databases has been extensively studied in recent years. This section reviews prior work organized by research focus: general database comparisons, e-commerce-specific studies, and methodological frameworks.

2.1. General PostgreSQL and MongoDB Comparisons

Several studies have compared the performance of PostgreSQL and MongoDB across various contexts. Jung et al. [8] examined input/output performance in big data environments and found that each system exhibited distinct strengths depending on the operation type. Güney and Ceylan [9] compared response times in specific test scenarios, demonstrating performance variations based on query complexity. Costa et al. [10] investigated the performance impact of database encryption on both systems, providing insights into security-performance trade-offs.

Cheng et al. [5] conducted an experimental comparison using unstructured GeoTIFF data, finding that MongoDB excelled for rapid storage of unstructured data while PostgreSQL performed better with smaller, structured datasets. Makris et al. [6] compared the two systems using 146 million AIS maritime traffic records, demonstrating that PostgreSQL achieved superior response times for complex spatio-temporal queries. These findings suggest that database performance advantages are highly context-dependent.

2.2. E-Commerce Database Studies

Database selection for e-commerce applications has received specific attention. Shen [11] conducted a comprehensive evaluation across various e-commerce scales, showing that while NoSQL databases offer flexibility for managing product catalogs, their performance varies significantly with data volume and query complexity. Similarly, Thirupattur et al. [12] analyzed SQL and NoSQL performance for optimizing retail operations, identifying query response time as a critical factor affecting user experience. Empirical studies focusing on large-scale e-commerce data further highlight the context-dependent nature of database performance. Aboutorabi et al. [13] evaluated SQL and MongoDB in e-commerce environments, reporting that MongoDB performs efficiently for handling unstructured data but exhibits latency issues in complex relational queries. According to Abbas and Farah [14] and Nakhare [15], effective e-commerce database optimization requires a balance between

the strong consistency guarantees of relational databases and the horizontal scalability of NoSQL systems. Despite these contributions, a recurring limitation in the existing literature is the lack of standardized experimental evaluations using strictly identical datasets and comparable query workloads. Many prior studies employ different data models or heterogeneous testing environments, which complicates the isolation of architectural performance factors. This study addresses this gap by applying a controlled experimental design with complete data parity and a focus on filtering and analytical workloads representative of typical e-commerce scenarios.

2.3. Methodological Frameworks and Literature Reviews

Taipalus [1] conducted a systematic literature review of database performance comparisons, noting that many existing studies lack methodological transparency and do not reflect real-world use cases. Khan et al. [2] provided a comprehensive review of the architectural differences between SQL and NoSQL, emphasizing the need for empirical studies in specific business contexts. Hassan [3] discussed considerations for selecting databases for big data applications.

2.4. Research Gap and Contribution

Table 1 summarizes the characteristics of prior studies compared with those of the present research.

Table 1. Comparison of the present study with prior research on PostgreSQL and MongoDB performance.

Study	Year	Data Type	Operations Tested	Dataset Scale	E-Commerce Focus	Identical Datasets
Cheng et al. [5]	2019	Unstructured (GeoTIFF)	Storage, retrieval	Small	No	N/A
Makris et al. [6]	2021	Spatio-temporal (AIS)	Complex queries	146 M records	No	Yes
Jung et al. [8]	2015	Big data	I/O performance	Large	No	Yes
Güney & Ceylan [9]	2022	Structured	Response times	Medium	No	Not specified
Costa et al. [10]	2022	Encrypted data	CRUD operations	Small	No	Yes
Shen [11]	2022	E-commerce	CRUD operations	Variable	Yes	Not specified
Aboutorabi et al. [13]	2015	E-commerce	Various	Large	Yes	Not specified
Thirupattur et al. [12]	2024	Retail/E-commerce	Various	Variable	Yes	Not specified
This study	2025	Structured transactional	Filtering, analytics	5 K–300 K products	Yes	Yes

Unlike prior studies that either focus on non-e-commerce contexts [5,9,11] or lack methodological transparency regarding dataset equivalence [14], this study provides: (1) a controlled comparison using identical datasets generated from a unified data model; (2) query scenarios explicitly mapped to OLTP/OLAP workload categories; (3) transparent statistical validation addressing the methodological gaps identified by Taipalus [1].

3. E-Commerce Data Characteristics

E-commerce activities are inseparable from various data flows related to product presentation, customer service, and order fulfilment. To understand what data are required and to estimate their volume, an analysis was conducted of publicly available information from popular online stores operating in Lithuania and abroad (e.g., pigu.lt, eavalyn.lt, worldofbooks.com). Based on this information, several categories of data can be distinguished:

- Product information, including product name, manufacturer, category, image, price, quantity, and descriptive attributes. The number and type of attributes depend on the product category.
- Customer data, collected when creating an account and typically including first name, last name, contact information, and purchase history.
- Order and payment information, including order date, order number, items purchased, quantities, prices, discounts, payment and delivery methods, and order status.
- Reviews and ratings, allowing users to evaluate purchased products and the order fulfillment process.
- User behavior tracking, collecting behavioral data to provide personalized content.

In summary, product, customer, and order information is generally strictly structured and can be stored in tables with clearly defined relationships. This trend is evident not only in real-world online store catalogues but also in publications on e-commerce data analysis and personalization.

4. Database Model Design

When developing the logical database model for this study, the design is based on both the analysis of existing online stores and the conceptual framework proposed by Song I. and Whang K. [16]. Their e-commerce data management architecture identifies several core data domains: users, products, inventory management, orders, and payments. To ensure comprehensiveness, additional data elements related to shopping carts, product categories, and customer reviews are also included.

The logical model (Figure 1) includes main entities and relationships. Each product is associated with a manufacturer (Brand) and can be assigned to one or more categories (Category). The category structure is hierarchical—each category may have a parent category. Each customer (Customer) may have multiple orders, forming a 1:N relationship. Each order (Order) can include more than one product, stored as separate OrderItem entities. Customers may also submit reviews for products they have purchased.

The presented model is applied in both PostgreSQL and MongoDB. In the relational database design, the logical model is refined according to normalization rules, meaning data are split into separate entities so that each table contains information about a single object or concept. Relationships are implemented using foreign keys. For example, the product–category many-to-many relationship is implemented through an additional ProductCategory table.

In MongoDB, a denormalized approach is used, meaning specific data (such as order items or addresses) are stored as embedded documents. This improves access speed and reduces the number of required queries for simple lookups.

Figure 2 illustrates the MongoDB document schema, showing how related data are embedded within a single document to optimize read performance for typical access patterns.

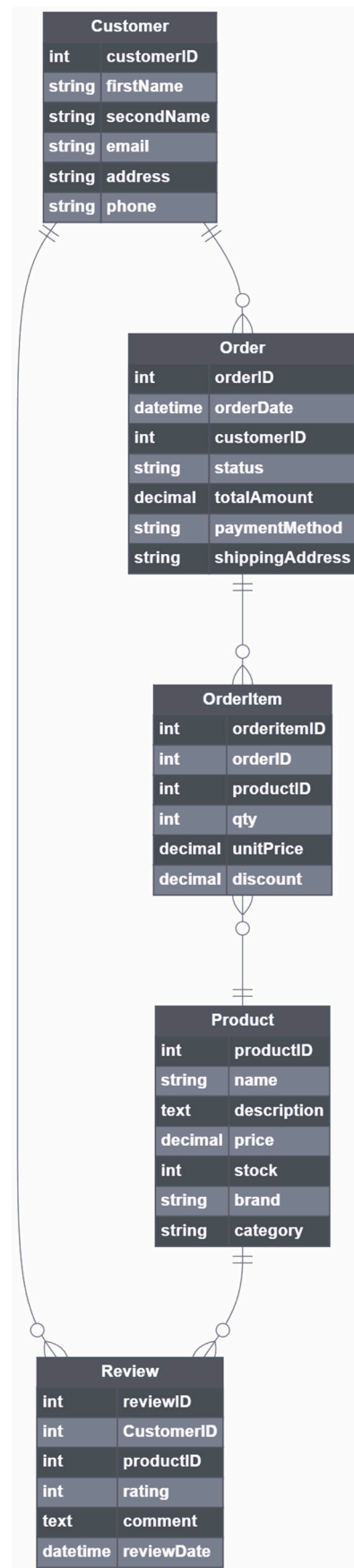


Figure 1. Conceptual data model for an e-commerce application.

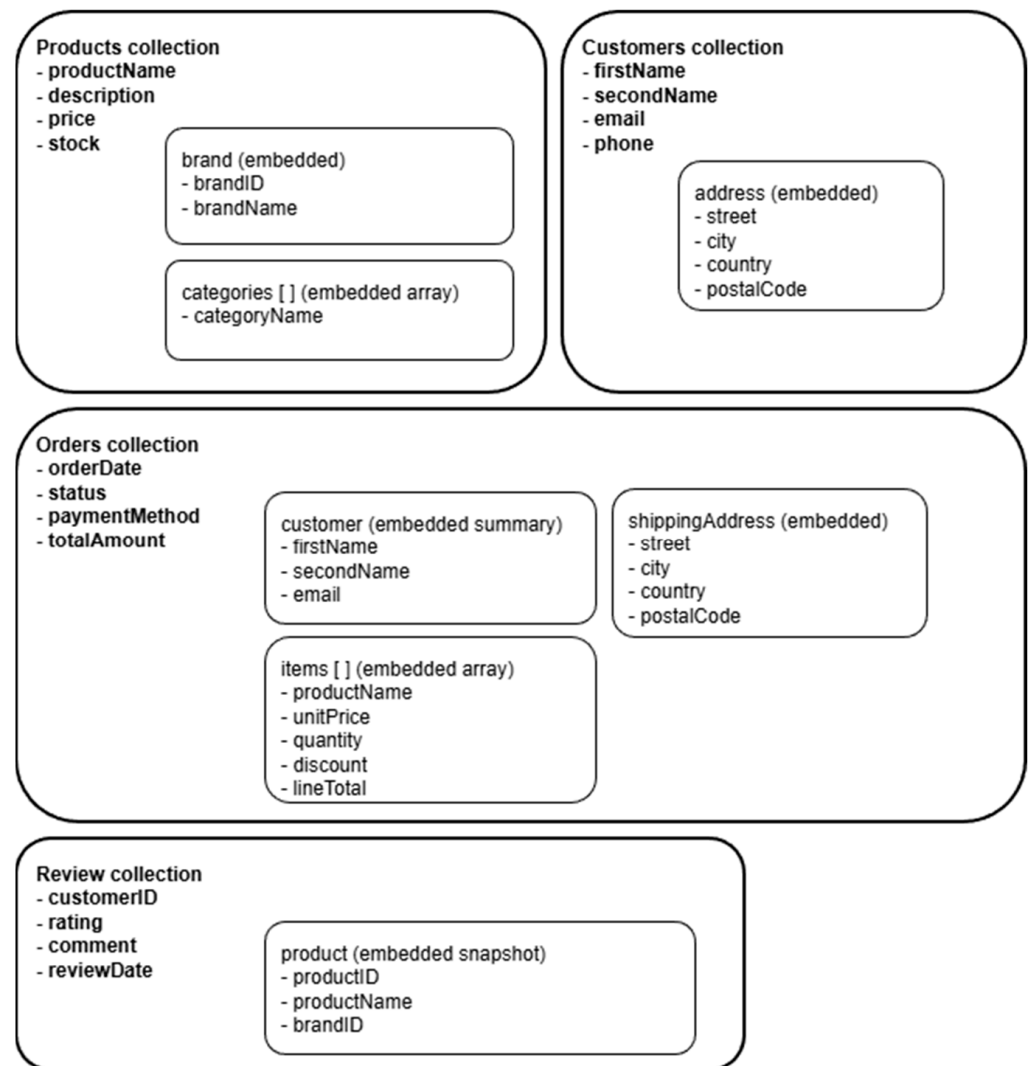


Figure 2. MongoDB document schema structure showing embedded documents for customer address, order items, and product categories.

5. Experimental Setup and Methodology

5.1. Hardware and Software Configuration

All experiments are conducted on the same computer to ensure identical conditions. Both database servers are installed and run locally (on localhost), thereby eliminating the influence of network-related factors. The tests are performed on a personal computer equipped with:

- AMD Ryzen 5 2500U processor (4 cores, 8 threads, base clock 2.0 GHz)
- 8 GB DDR4 RAM
- SSD storage (SATA III interface)
- Windows 10 Home 64-bit operating system

Database management systems used:

- PostgreSQL version 17.5 with default configuration (shared_buffers = 128 MB, effective_cache_size = 4 GB, work_mem = 4 MB)
- MongoDB version 7.0.14 with WiredTiger storage engine and default cache size (50% of RAM minus 1 GB = approximately 3 GB)

Important limitation: The test machine's limited hardware resources (8 GB RAM) mean these results emphasize relative performance differences between the two database systems

rather than absolute execution times. Production environments with greater resources may yield different absolute performance values, though relative trends are expected to remain consistent.

The evaluation of non-indexed data follows the staged evaluation methodology recommended in the database benchmarking literature [1,7]. This approach isolates core query execution engine behavior from index-dependent access path selection. Specifically, it measures the efficiency of sequential scans, join algorithms, and aggregation pipelines in their baseline state. This approach enables attribution of observed performance differences to fundamental architectural characteristics rather than index configuration choices, establishing a reference point against which indexed performance can subsequently be compared. A minimal indexed validation scenario (Section 6.5) empirically confirms that the baseline trends remain consistent under simple indexing configurations.

Configuration note: Both systems were tested using their default configurations without optimization for specific workloads. This approach ensures fair comparison but means that properly tuned systems could achieve better absolute performance. The study intentionally avoids system-specific optimizations to maintain objectivity and reproducibility.

Following the YCSB standard [7], the first two query executions were excluded from analysis to eliminate cold-start effects and ensure steady-state performance measurements.

5.2. Data Generation

To populate both databases with data, a single Python 3.12.3 script generates and inserts identical datasets into the relational and document-oriented databases, ensuring a fair and accurate performance comparison. Synthetic data are generated using the Faker library, which allows automated creation of realistic-looking data. For transparency and reproducibility, the complete datasets, data generation scripts, and raw benchmarking results are provided in the Supplementary Materials (Zenodo repository, <https://doi.org/10.5281/zenodo.18084839>).

The study is carried out in three stages, with progressively increasing data volumes, to observe how both systems behave as data volume increases. Table 2 presents the data volumes used in each stage.

Table 2. Data Volumes used in the study.

Stage	Products	Categories	Brands	Customers	Streets	Cities	Countries	Orders	Reviews	Order Item Rows
I	5000	25	130	5000	250	100	10	9000	30,000	49,474
II	100,000	250	1000	100,000	1000	500	100	100,000	100,000	549,417
III	300,000	250	2000	300,000	2000	1000	250	300,000	300,000	1,646,921

5.3. Query Design and Testing Methodology

To evaluate performance differences between relational and document-oriented database management systems, a set of test scenarios was formulated to represent the most common operations in e-commerce systems. The focus is on read operations (SELECT queries) representing both typical customer filtering behaviour and analytical operations performed by staff.

Six testing queries were selected (identified as select_test1 through select_test6).

Following established workload characterization principles from benchmarking theory [8], the queries are categorized into two groups based on their operational characteristics:

OLTP-style filtering queries (select_test1–select_test3): These queries simulate typical customer behavior, including point lookups and range scans across single or multiple

criteria. They represent the interactive, latency-sensitive operations that occur every day in e-commerce front-end systems.

OLAP-style analytical queries (select_test4–select_test6): These queries involve aggregations, multi-table joins, and complex filtering conditions typical of business intelligence and reporting workloads. They represent back-office analytical operations for inventory management, customer analytics, and order processing.

This classification enables interpretation of results within the context of established database workload theory, where relational systems are traditionally optimized for OLAP workloads. In contrast, document databases may offer advantages for simple OLTP-style document retrieval.

Filtering queries simulating real customer behavior:

- select_test1—customer search based on a single criterion (e.g., city)
- select_test2—product search based on two criteria (e.g., brand and price range)
- select_test3—product search based on three or more criteria (e.g., price, category, stock availability)

Analytical queries for business intelligence:

- select_test4—customers and their orders whose total value exceeded a specified threshold within a given period
- select_test5—products that received at least one review with a rating lower than 3
- select_test6—orders placed by customers from country X where a specific payment method was used

Important note on query equivalence: Due to fundamental structural differences between relational and document-oriented data models, queries select_test3 and select_test6 return slightly different result set sizes. In PostgreSQL’s normalized structure, products associated with multiple categories appear multiple times in join results, whereas MongoDB’s denormalized model with embedded category arrays returns each product once. This structural difference is inherent to the data models and does not invalidate the performance comparison, as both queries retrieve semantically equivalent business information. The performance analysis, therefore, focuses on relative execution efficiency rather than on absolute result-set matching.

Table 3 presents the SQL and MongoDB query syntax for all six experimental scenarios. The SQL queries correspond to the relational normalized data model, whereas the MongoDB queries are designed for the document-based denormalized model using embedded objects and aggregation pipelines.

Table 3. Testing queries.

Short Query Name	SQL	MongoDB
select_test1	<pre>SELECT c.customerID, c.firstName, c.secondName, c.email, ci.cityName FROM customer c INNER JOIN address a ON c.addressID = a.addressID INNER JOIN street s ON a.streetID = s.streetID INNER JOIN city ci ON s.cityID = ci.cityID WHERE ci.cityName = 'Port John';</pre>	<pre>db.customers.find({ "address.city": "Port John" })</pre>

Table 3. Cont.

Short Query Name	SQL	MongoDB
select_test2	<pre> SELECT p.productID, p.productName, p.price, b.brandName FROM product p INNER JOIN brand b ON p.brandID = b.brandID WHERE b.brandName like 'A%' AND p.price between 100 and 300; </pre>	<pre> db.products.find({ "brand.brandName": { \$regex: /^A/ }, "price": { \$gte: 100, \$lte: 300 } }) </pre>
select_test3	<pre> SELECT p.productID, p.productName, p.stock, b.brandName, c.categoryName FROM product p INNER JOIN brand b ON p.brandID = b.brandID INNER JOIN productcategory pc ON p.productID = pc.productID INNER JOIN category c ON pc.categoryID = c.categoryID WHERE b.brandName between 'E' and 'M' AND c.categoryName in ('Under', 'Side', 'Unit', 'After', 'Control') AND p.stock > 10; </pre>	<pre> db.products.find({ stock: { \$gt: 10 }, "brand.brandName": { \$gte: "E", \$lte: "M" }, "categories.categoryName": { \$in: ["Under", "Side", "Unit", "After", "Control"] } }, { productName: 1, stock: 1, "brand.brandName": 1, "categories.categoryName": 1 }) </pre>
select_test4	<pre> SELECT c.customerID, c.firstName, c.secondName, o.orderID, o.orderDate, SUM(oi.unitPrice * oi.quantity) AS order_total FROM customer c INNER JOIN customerorder o ON c.customerID = o.customerID INNER JOIN orderitem oi ON o.orderID = oi.orderID WHERE o.orderDate BETWEEN '2024-01-01' AND '2024-12-31' GROUP BY c.customerID, c.firstName, c.secondName, o.orderID, o.orderDate HAVING SUM(oi.unitPrice * oi.quantity) > 300; </pre>	<pre> db.orders.aggregate([{ "\$match": { "orderDate": { "\$gte": datetime(2024, 1, 1), "\$lte": datetime(2024, 12, 31) } }}, { "\$unwind": "\$items" }, { "\$group": { "_id": "\$_id", "orderDate": { "\$first": "\$orderDate" }, "customer": { "\$first": "\$customer" }, "order_total": { "\$sum": { "\$multiply": ["\$items. unitPrice", "\$items.quantity"] } } }}, { "\$match": { "order_total": { "\$gt": 300 } }}, { "\$project": { "orderID": "\$_id", "orderDate": 1, "order_total": 1, "firstName": "\$customer.firstName", "secondName": "\$customer.secondName" } }]) </pre>

Table 3. Cont.

Short Query Name	SQL	MongoDB
select_test5	<pre> SELECT DISTINCT p.productID, p.productName FROM product p INNER JOIN review r ON p.productID = r.productID WHERE r.rating < 3; </pre>	<pre> db.review.aggregate([{ "\$match": { "rating": { "\$lt": 3 } } }, { "\$lookup": { "from": "products", "localField": "productID", "foreignField": "productID", "as": "product" } }, { "\$unwind": "\$product" }, { "\$project": { "_id": 0, "productID": "\$product.productID", "productName": "\$product.productName" } }, { "\$group": { "_id": "\$productID", "productName": { "\$first": "\$productName" } } }]) </pre>
select_test6	<pre> SELECT o.orderID, o.orderDate, c.customerID, c.firstName, c.secondName, co.countryName, pm.methodName, o.totalAmount FROM customerorder o INNER JOIN customer c ON o.customerID = c.customerID INNER JOIN address a ON c.addressID = a.addressID INNER JOIN city ci ON ci.cityID = a.cityID INNER JOIN country co ON co.countryID = ci.countryID INNER JOIN paymentmethod pm ON o.paymentMethodID = pm.methodID WHERE pm.methodName = 'PayPal' AND co.countryName = 'Estonia' ORDER BY o.orderDate DESC; </pre>	<pre> db.orders.find({ "paymentMethod.methodName": "PayPal", "customer.address.country": "Estonia" }) </pre>

During testing, each query is executed 15 times to ensure reliability and minimize the influence of random system factors, in line with standard database benchmarking practices. This approach follows established database benchmarking methodologies, such as the YCSB (Yahoo! Cloud Serving Benchmark) framework proposed by Cooper et al. [7], which emphasizes the importance of multiple repetitions for statistical validity. The choice of

15 repetitions balances statistical reliability with practical testing time constraints for this scale of study.

All tests are performed on non-indexed data to establish baseline performance characteristics under explicitly bounded assumptions. This design decision isolates core execution-engine behavior and query optimization capabilities without the confounding influence of indexing strategies. A minimal indexed validation scenario (Section 6.5) confirms that the baseline trends remain consistent under simple indexing configurations. Execution times and outputs are automatically recorded using scripts. For every execution, the query runtime is measured in milliseconds, and the final results are presented as the average execution time and standard deviation.

Warm-up procedure: To ensure fair comparison and eliminate cold-start effects, the first two executions of each query are excluded from measurements. This allows both database systems to populate their caches and complete JIT compilation, ensuring that recorded measurements reflect steady-state performance rather than initialization overhead.

The number of records returned by each query is also logged to verify query equivalence and assess how result set size influences performance. The read-operation tests are repeated with different data volumes (Stages II and III) to evaluate how the databases behave as datasets grow.

6. Results

6.1. Storage Space Requirements

The first observation made after creating the databases with identical data is the difference in storage size. With Stage I data, the MongoDB database occupies approximately four times less space than PostgreSQL (5.06 MB and 21.03 MB, respectively). In Stage II, this difference decreases: the MongoDB size is 31% smaller (93.38 MB compared to 135.32 MB). In Stage III, where the amount of data in the main tables/documents is tripled, the size difference remains similar: the MongoDB database is approximately 33% smaller, at 249.66 MB, compared to 374.96 MB for PostgreSQL.

These differences arise from distinct storage architectures and mechanisms used by each system. MongoDB's document-oriented model allows related data to be embedded within a single document, reducing data duplication and improving compression efficiency. PostgreSQL, as a relational database, stores data in normalized tables connected through relationships and indexed for integrity and query execution. The observed storage difference is therefore primarily related to structural characteristics of relational storage, including tuple metadata, fixed-size page allocation and alignment, normalization overhead and supporting indexes.

Since the dataset is generated once and subsequently used in a read-only benchmarking workload, MVCC version accumulation does not significantly affect the measured storage size in this experiment.

The reported values represent the total physical database size rather than the logical data volume.

6.2. Query Result Set Verification

Since comparing execution times is meaningful only when both database systems return semantically equivalent results, the number of rows returned by each query was assessed prior to conducting performance analysis. Table 4 presents these results.

Table 4. Number of rows returned by the queries.

QueryType	Stage I Rows		Stage II Rows		Stage III Rows	
	PostgreSQL	MongoDB	PostgreSQL	MongoDB	PostgreSQL	MongoDB
select_test1	56	56	320	320	440	440
select_test2	123	123	2228	2228	6011	6011
select_test3 *	424	426	216	210	5467	5198
select_test4	5475	5475	59,846	59,846	179,907	179,907
select_test5	4556	4556	32,941	32,941	70,070	70,070
select_test6 *	183	181	633	620	402	430

* Note: Queries marked with an asterisk show minor differences caused by normalized relational joins versus embedded document representation. Differences reflect native model semantics rather than inconsistencies in query logic. Bold rows indicate cases where differences in returned row counts were observed between PostgreSQL and MongoDB.

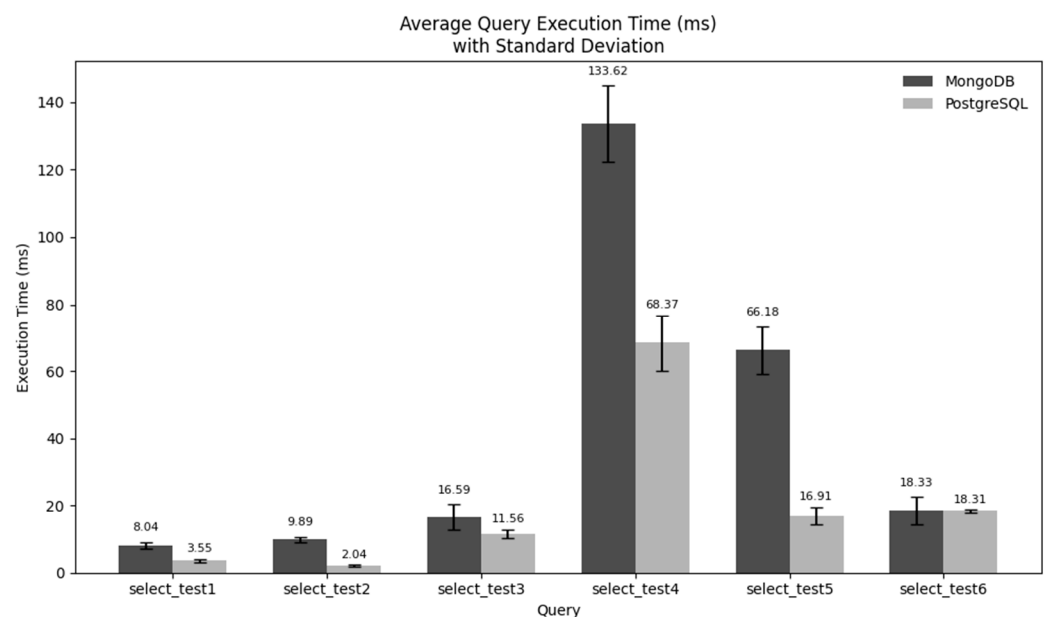
After verifying the results, it was determined that the number of rows returned by most queries in MongoDB and PostgreSQL matched. Differences were observed only in queries select_test3 and select_test6.

The observed row count differences originate from the semantic level at which results are produced in each data model. In the normalized relational schema, joins return one row per relationship instance, while in the document model embedded data produce one document per entity. Using DISTINCT would alter the relational query semantics and introduce an additional aggregation step. Therefore, the comparison evaluates execution behavior under native model semantics.

6.3. Query Performance Analysis

6.3.1. Stage I Results (Small Dataset)

Figure 3 presents query execution results for operations performed on the smallest dataset (Stage I). Query execution times vary with query complexity. The results indicate that both database management systems perform similarly for simpler filtering queries (those with 1–2 selection conditions)—execution times are relatively low, and differences between PostgreSQL and MongoDB are minimal.

**Figure 3.** Average query execution times with standard deviation (Stage I).

However, when moving to more complex analytical queries, differences become more noticeable: PostgreSQL maintains more stable execution times with lower standard deviation, whereas MongoDB shows greater variation in query performance. This may indicate higher sensitivity of the system to specific data structures or processing scenarios.

6.3.2. Stage II Results (Medium Dataset)

Figure 4 illustrates how query execution times change as the number of records in the databases increases, with data volumes in Stage II ranging approximately from $3\times$ (reviews) to $20\times$ (products and customers) compared to Stage I.

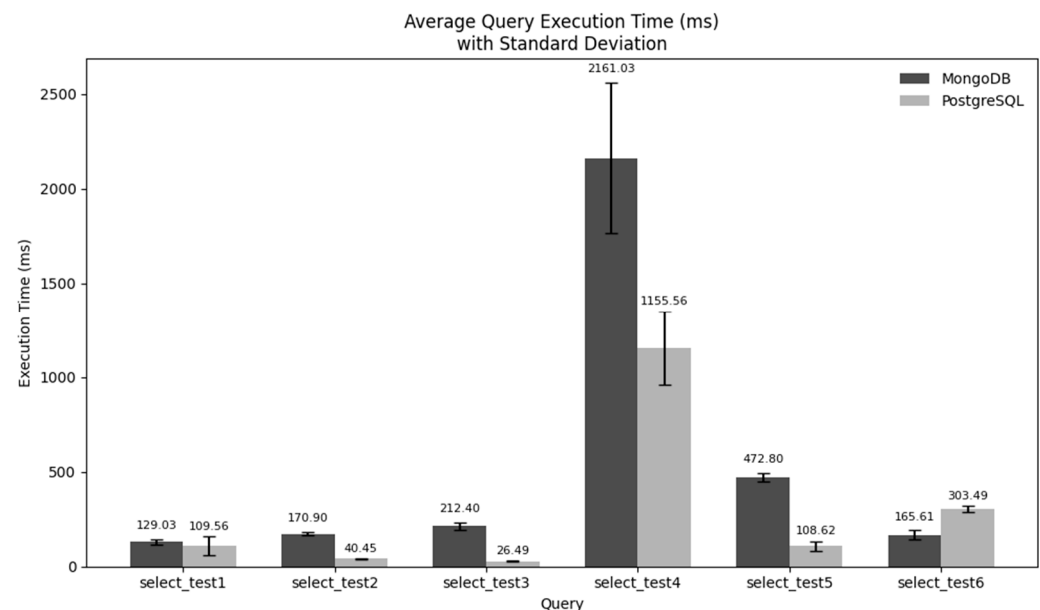


Figure 4. Average query execution times with standard deviation (Stage II).

The results show that query execution times increase in both systems; however, the magnitude of the increase is not proportional across all queries. For simpler filtering queries with 1–3 criteria, PostgreSQL performs significantly better—execution times are 2 to 5 times shorter than MongoDB, and time variations are also smaller. In analytical queries, the differences become even more pronounced: PostgreSQL processes joins and aggregations more efficiently, while MongoDB produces slower, less stable results.

6.3.3. Stage III Results (Large Dataset)

Figure 5 illustrates how query execution times change when the number of records is increased further, with data volumes in Stage III ranging approximately from $8\times$ (streets) to $60\times$ (products and customers) compared to Stage I.

The results show that as data volume increases, query execution times rise in both systems; however, the differences between PostgreSQL and MongoDB become even more pronounced. As in the previous stages, filtering queries in the relational database are executed significantly faster and exhibit a lower standard deviation, indicating more stable performance. A similar trend is observed for more complex analytical queries: PostgreSQL achieves shorter execution times and lower variability, whereas MongoDB not only produces slower results but also displays a wider distribution of execution times.

When evaluating all stages together, it becomes evident that select_test4 (customers and their orders whose total amount exceeded a specified threshold within a given period) consistently takes longer to execute across both database management systems stages. This is due to the nature of the query—it requires calculations involving joins across multiple

tables/documents and additional filtering based on an aggregated metric. Nevertheless, even in this scenario, PostgreSQL demonstrates better performance and more stable results.

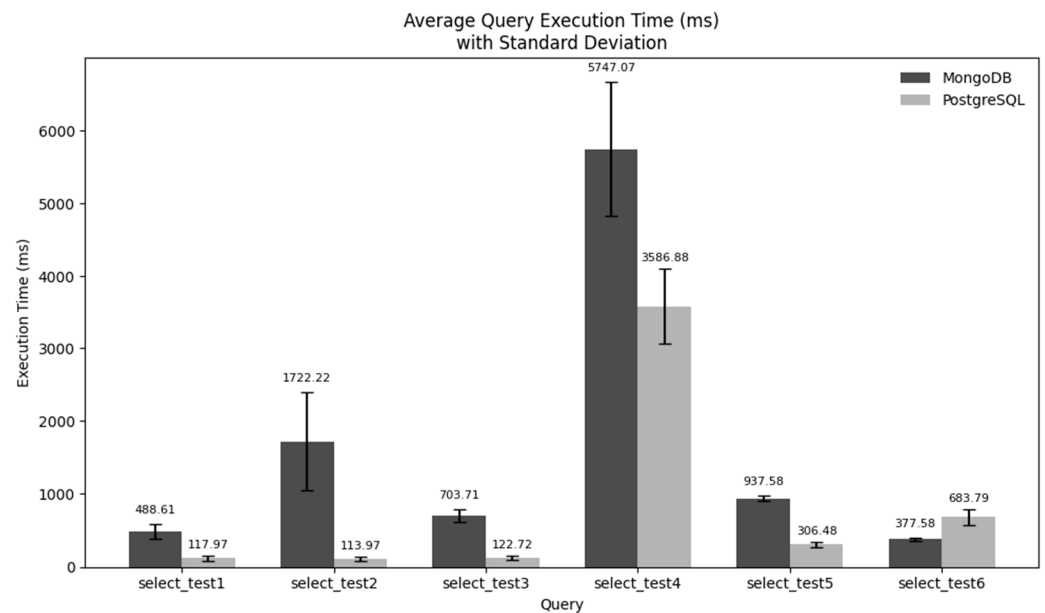


Figure 5. Average query execution times with standard deviation (Stage III).

6.4. Statistical Significance Testing

To determine whether the observed performance differences between PostgreSQL and MongoDB are statistically significant, paired *t*-tests were conducted for each query type across all three experimental stages. The null hypothesis (H_0) states that there is no difference in mean execution times between the two database systems, i.e., $\mu_{PG} - \mu_{MG} = 0$. For each query type, paired execution attempts were aligned by attempt index, and the difference distribution was analyzed using the paired *t*-test. Effect sizes were quantified using Cohen's *d*, calculated as the standardized mean difference in paired observations.

Across all three stages, the results indicate statistically significant performance differences for most query types (detailed statistical calculations are provided in the Supplementary Materials Zenodo repository, <https://doi.org/10.5281/zenodo.18084839>). PostgreSQL consistently outperformed MongoDB across select_test1–select_test5, whereas MongoDB demonstrated significantly faster execution for select_test6 in Stages II and III, with no significant difference in Stage I. Most tests yielded $p < 0.001$; the exceptions were select_test6 in Stage I and select_test1 in Stage II, where performance differences were not statistically significant. Effect sizes were large ($|d| > 1$) or extremely large for the majority of queries, confirming strong practical significance.

For Stage III, which represents the largest dataset and highest system load, the following results were obtained:

- select_test1: $t(12) = -11.81, p < 0.001, d = -3.27$
- select_test2: $t(12) = -8.16, p < 0.001, d = -2.26$
- select_test3: $t(12) = -5.15, p < 0.001, d = -1.43$
- select_test4: $t(12) = -6.05, p < 0.001, d = -1.68$
- select_test5: $t(12) = -44.5, p < 0.001, d = -12.34$
- select_test6: $t(12) = 9.23, p < 0.001, d = 2.56$

These results provide strong statistical evidence that the performance differences observed between the two systems are not due to random variation but reflect consistent, systematic characteristics of their execution models. PostgreSQL exhibits a substantial and widening performance advantage for relational-style filtering and aggregation queries

as data volume increases, whereas MongoDB demonstrates superior performance for the specific workload represented by `select_test6`. The very large effect sizes across nearly all queries show that the measured differences are not only statistically significant but also practically meaningful for engineering decision-making.

To complement the statistical analysis, Table 5 presents performance ratios, calculated as the ratio of the mean MongoDB execution time to the mean PostgreSQL execution time for each query type and experimental stage. Values greater than 1.0 indicate faster execution in PostgreSQL, while values below 1.0 indicate superior performance in MongoDB.

Table 5. Precise Performance Ratios by Query Type and Stage (MongoDB execution time/PostgreSQL execution time).

Query	Workload Category	Stage I	Stage II	Stage III
<code>select_test1</code>	OLTP filtering	2.26	1.18	4.14
<code>select_test2</code>	OLTP filtering	4.84	4.22	15.11
<code>select_test3</code>	OLTP filtering	1.43	8.02	5.73
<code>select_test4</code>	OLAP analytical	1.95	1.87	1.6
<code>select_test5</code>	OLAP analytical	3.91	4.35	3.06
<code>select_test6</code>	OLAP analytical	1.00	0.55 *	0.55 *

* Values below 1.0 indicate MongoDB outperformed PostgreSQL for this specific query type. Note: Ratios calculated from mean execution times across 15 repetitions (excluding 2 warm-up runs). The overall weighted average across all queries and stages is $3.65\times$, with PostgreSQL demonstrating consistent advantages for 5 of 6 query types.

6.5. Indexed Configuration Validation

To validate whether the baseline findings remain consistent under minimal indexing assumptions, a supplementary experiment was conducted using the Stage III dataset. This validation addresses the theoretical concern that performance conclusions might be substantially altered by index presence.

Query selection rationale: Two query types were selected to represent both major workload categories identified in Section 5.3:

- `select_test2` (OLTP-style filtering query): Multi-criteria product search on brand and price range. This query represents typical customer browsing behavior and was selected because it exhibits the most pronounced PostgreSQL performance advantage among filtering queries ($15.1\times$ faster in Stage III, Table 5). Its compound filter criteria (brand name + price range) directly align with the composite index structure, making it an ideal candidate to assess whether indexing alters the observed baseline patterns.
- `select_test4` (OLAP-style analytical query): Customer order aggregation involving temporal filtering, multi-table joins, and GROUP BY/HAVING clauses. This query represents back-office business intelligence operations and was selected as the most complex analytical query in the test suite, requiring join operations across customer, order, and order item entities with aggregated metric filtering. It exhibited the longest absolute execution times across both systems (Stage III: PostgreSQL 3537.88 ms, MongoDB 5747.07 ms), making it representative of computationally intensive analytical workloads.

This two-query selection provides a representative sample spanning both OLTP and OLAP workload categories while maintaining practical testing constraints.

For each query and database system, a minimal indexing configuration was applied, consisting of one composite secondary index per query in addition to mandatory primary keys:

PostgreSQL:

- CREATE INDEX idx_product_brandid_price ON product (brandID, price);
- CREATE INDEX idx_customerorder_orderdate ON customerorder (orderDate);

MongoDB:

- db.products.createIndex({"brand.brandName": 1, "price": 1}, {name: "idx_brand_price"});
- db.orders.createIndex({"orderDate": 1}, {name: "idx_orderdate"});

The indexed validation followed the same experimental methodology as the baseline: 15 executions per query, warm-up exclusion of the first two attempts, and measurement in the identical hardware and software environment. Because the dataset was independently regenerated, absolute execution times are not compared to the main baseline results. Instead, the analysis focuses exclusively on within-dataset relative performance changes between non-indexed and indexed configurations.

Table 6 presents the validation results.

Table 6. Indexed validation results showing relative performance change (Stage III dataset).

Query	DBMS	Baseline Mean (ms)	Baseline Std. Dev. (ms)	Indexed Mean (ms)	Indexed Std. Dev. (ms)	Relative Change (%)
select_test2	PostgreSQL	113.68	40.03	116.84	41.41	−2.78
select_test2	MongoDB	83.99	12.44	83.99	12.44	0.00
select_test4	PostgreSQL	1365.12	96.29	1362.14	86.20	0.22
select_test4	MongoDB	3718.71	299.24	3678.14	317.08	1.09

The results demonstrate that minimal indexing does not substantially alter the performance trends observed in the baseline experiment. For select_test2, PostgreSQL exhibited a marginal performance degradation of 2.78%, while MongoDB showed no measurable change. For select_test4, both systems showed minimal improvement (PostgreSQL 0.22%, MongoDB 1.09%). These negligible relative changes confirm that the baseline conclusions—particularly PostgreSQL’s systematic performance advantages for analytical queries—remain valid under simple indexing scenarios.

This validation should not be interpreted as a comprehensive analysis of indexing strategies, which remains outside the scope of this study. Rather, it serves to empirically support the robustness of the baseline findings and confirm that the main conclusions are not fundamentally invalidated by the absence of secondary indexes in the primary experimental design. The fact that minimal indexes provide negligible performance improvement suggests that PostgreSQL’s advantages stem from fundamental architectural characteristics of its query execution engine rather than from differences in index utilization.

7. Discussion

The experimental findings demonstrate that the choice between relational and document-oriented databases for e-commerce applications should be guided primarily by query complexity and data volume rather than storage efficiency considerations. PostgreSQL’s 1.6–15.1× faster execution times for both analytical queries involving joins and aggregations indicate that normalized relational schemas remain the optimal choice for transactional e-commerce systems where order processing, inventory management, and customer analytics dominate the workload.

7.1. Comparison with the Existing Literature

These findings align with and extend the work of Makris et al. [6], who demonstrated PostgreSQL's superior performance for complex spatio-temporal queries on large-scale maritime AIS data. While their study focused on geospatial operations with 146 million records, the present research demonstrates similar performance patterns at the scale of small- to medium-sized e-commerce platforms (up to 300,000 products), suggesting that PostgreSQL's advantages for complex analytical workloads are consistent across different application domains and dataset sizes.

Similarly, Taipalus's systematic literature review [1] emphasized that database selection should not rely solely on performance metrics. The present study validates this perspective by demonstrating that storage efficiency (MongoDB's 33% space advantage) does not compensate for performance degradation in query-intensive e-commerce scenarios. This finding addresses Taipalus's [1] call for more empirical studies that reflect real-world use cases rather than theoretical comparisons.

The results also complement the work of Cheng et al. [5], who found MongoDB efficient for rapid storage of unstructured GeoTIFF data. The present study demonstrates that for structured e-commerce data with complex relational queries, the performance trade-offs differ significantly. While Cheng et al. [5] identified scenarios where MongoDB excels (unstructured data, simple storage operations), this research identifies where relational databases maintain clear advantages (structured data, complex analytical queries).

Recent comparative studies by Taipalus [1], Yedilkhan et al. [17], and Carvalho et al. [18] have highlighted the importance of query complexity, data structures, and indexing strategies for database performance. The present study contributes to this body of literature by providing a controlled comparison using identical datasets and standardized e-commerce scenarios. The finding that PostgreSQL maintains performance advantages even without indexing suggests that its query optimization engine is particularly well-suited for relational e-commerce workloads.

Kuszera et al. [19] explored data structure alternatives in RDB to NoSQL document store conversion, noting that schema flexibility comes with performance trade-offs. The present findings quantify these trade-offs in the specific context of e-commerce, demonstrating that MongoDB's schema flexibility does not compensate for its slower query execution when handling structured transactional data with complex analytical requirements.

7.2. Practical Implications

The observed stability (lower standard deviation) in PostgreSQL's execution times further suggests that PostgreSQL provides more predictable response times—a critical factor for user experience in online retail environments. MongoDB's 30–35% storage advantage, while notable, does not compensate for performance degradation at scale.

However, this storage-performance trade-off may be acceptable in scenarios where product catalogs require frequent schema modifications or where semi-structured data (such as diverse customer reviews with varying attributes) predominate. For small- and medium-sized e-commerce platforms processing up to 300,000 products with predominantly read-heavy analytical workloads, PostgreSQL is the recommended database system.

MongoDB should be considered primarily when schema flexibility requirements outweigh query performance demands. This might occur in scenarios such as:

- Rapidly evolving product catalogs where attribute structures change frequently
- E-commerce platforms emphasizing simple product lookups over complex analytical reporting
- Systems where storage costs are prohibitive, and query performance is less critical

It is important to emphasize that these conclusions are limited to single-host deployments without indexing. Distributed configurations and indexed queries may yield different trade-offs requiring further investigation.

7.3. Limitations and Scope

Several methodological and contextual limitations should be acknowledged when interpreting the results of this study. First, all experiments were run on a single machine with 8 GB of RAM, which limits absolute throughput and may amplify relative performance differences between the two systems. Consequently, the findings reflect comparative behavior under resource-constrained conditions rather than the full capabilities of each DBMS. In addition, both PostgreSQL and MongoDB were evaluated using their default configurations. While this supports reproducibility and aligns with typical out-of-the-box deployments found in small organizations, it does not account for potential performance improvements achievable through workload-specific tuning, such as optimized memory allocation, caching strategies, or aggregation pipelines.

The scope of the work was further limited by its focus on read-only workloads. Only SELECT queries were evaluated, meaning that write-intensive, update-heavy, or mixed transactional workloads—where concurrency control, logging mechanisms, and durability guarantees differ substantially between PostgreSQL and MongoDB—remain outside the study's scope. To establish a baseline, all experiments were conducted on non-indexed datasets. This decision isolates core execution-engine behavior but does not reflect production scenarios in which indexing strategies might influence absolute performance values. A minimal indexed validation scenario (Section 6.5) confirms that the baseline trends remain consistent under simple indexing configurations, though comprehensive indexing analysis remains outside the scope of this work and is recommended for future research.

Another limitation arises from the single-host deployment model. The experiments did not incorporate replication, sharding, or distributed execution, and thus do not capture performance characteristics that emerge in horizontally scaled or fault-tolerant architectures. Finally, minor discrepancies in result-set size were observed for `select_test3` and `select_test6` due to structural differences between normalized and denormalized data models in PostgreSQL and MongoDB. While the retrieved business information remained equivalent, these representational differences may introduce slight performance variations.

Despite these limitations, the study provides meaningful empirical insights for small- and medium-scale e-commerce environments, where deployments typically rely on structured data models, single-host configurations, and read-intensive workloads. The findings offer practical guidance for practitioners evaluating relational versus document-oriented database technologies under similar constraints.

7.4. Research and Practitioner Contributions

This study makes distinct contributions for both research and practitioner audiences:

Research contributions: (1) Controlled empirical validation of theoretical predictions regarding relational database advantages for join-intensive analytical workloads. (2) Methodological framework for database comparison studies emphasizing dataset equivalence, workload categorization, and statistical validation. (3) Quantified baseline performance data enabling future comparisons with indexed and distributed configurations.

Practitioner contributions: (1) Evidence-based decision guidelines for small and medium e-commerce platform development. (2) Quantified performance trade-offs between storage efficiency (MongoDB advantage) and query execution speed (PostgreSQL advantage). (3) Decision framework for database selection based on workload characteristics.

Table 7 provides a practical decision framework that synthesizes the study's findings.

Table 7. Database selection decision framework for e-commerce applications.

Decision Criterion	Favor PostgreSQL	Favor MongoDB
Primary workload	Complex analytical queries, reporting, aggregations	Simple document retrieval, catalog browsing
Data structure	Highly relational, normalized, stable schema	Variable attributes, frequently evolving schema
Query complexity	Multi-table joins, GROUP BY, and HAVING clauses	Single-collection lookups, embedded document access
ACID requirements	Strong consistency is critical for transactions	Eventual consistency acceptable
Storage budget	Flexible (higher storage acceptable)	Constrained (33% storage savings possible)
Response time predictability	Critical for user experience	Less important
Team expertise	SQL and relational modeling proficiency	JSON/document data modeling familiarity
Scaling strategy	Vertical scaling sufficient	Horizontal scaling (sharding) planned

8. Conclusions

This empirical study compared PostgreSQL and MongoDB read performance in e-commerce scenarios using identical datasets, standardized query patterns mapped to OLTP/OLAP workload categories, and controlled experimental conditions on resource-constrained single-host hardware typical of small- and medium-sized deployments. The key findings demonstrate that relational database architecture provides substantial advantages for structured e-commerce data with complex analytical requirements.

PostgreSQL consistently maintained $1.4\text{--}15.1\times$ faster execution times for multi-criteria queries and analytical operations involving joins and aggregations across all three dataset scales. The performance advantage increased with data volume, and PostgreSQL exhibited significantly lower standard deviation in execution times, indicating more predictable response characteristics. Statistical significance testing confirmed that these differences reflect systematic architectural advantages rather than random variation ($p < 0.001$ for all query types).

These findings directly support Hypothesis H1, confirming that PostgreSQL delivers superior performance for join-intensive analytical queries through its relational execution model and query optimization mechanisms.

MongoDB's 33% storage efficiency advantage, while measurable, does not compensate for the performance degradation in query-intensive scenarios typical of e-commerce platforms. MongoDB remained competitive only for simple filtering queries on smaller datasets (Stage I), but performance degraded substantially as data volumes increased and query complexity grew.

Accordingly, Hypothesis H2 is only partially supported: MongoDB demonstrates performance advantages for specific access patterns, but these benefits do not extend across the broader range of analytical and filtering workloads evaluated in this study.

For small- and medium-sized e-commerce platforms (up to 300,000 products) operating in single-host, budget-constrained environments with predominantly read-heavy analytical workloads, structured transactional data, and requirements for complex customer analytics and inventory reporting, PostgreSQL demonstrates substantial performance advantages for the workload profile tested in this study. These findings should not be generalized to indexed configurations, write-intensive workloads, or distributed deployment scenarios without further investigation. The normalized relational model's ability to

efficiently process joins, aggregations, and multi-table queries makes it well-suited for the data access patterns characteristic of online retail operations.

MongoDB should be considered primarily in scenarios where schema flexibility and rapid structural evolution outweigh query performance requirements. This might include product catalogues with highly variable attribute structures or systems emphasizing simple document retrieval over complex analytical reporting.

The experimental scope explicitly bounds these conclusions: single-host deployments with 8 GB RAM, non-indexed baseline datasets, read-only operations, and default system configurations. A minimal indexed validation scenario (Section 6.5) confirmed that the baseline trends remain consistent under simple indexing configurations, though comprehensive indexing strategies, write-intensive workloads, and distributed architectures require further investigation. Enterprise-scale deployments with greater hardware resources, production-optimized configurations, or distributed architectures may exhibit different performance characteristics. The observed patterns align with database architecture theory but require validation across broader deployment scenarios.

9. Future Work

Although this study focused on query execution times for read operations, future research should expand the analysis by including:

- Write operation performance: While Jung et al. [8] have examined write operations in general contexts, a comprehensive evaluation of INSERT, UPDATE, and DELETE operations specific to e-commerce transaction patterns (order processing, inventory updates) remains needed.
- Indexing strategies: While a minimal indexed validation (Section 6.5) confirmed the consistency of baseline findings under simple secondary indexes, systematic investigation of compound indexes, partial indexes, covering indexes, and index-organized tables could reveal optimization opportunities for specific query patterns. Building on the baseline established in this study, investigation of how B-tree, hash, and compound indexing strategies affect the observed performance patterns would quantify the practical improvements achievable in production deployments. Prior work [20] has demonstrated that indexing strategies can substantially influence database performance in resource-constrained environments. In addition, prior work has evaluated the insertion speed of indexed spatial data across MySQL, PostgreSQL, and MongoDB [21] highlighting the importance of indexing and motivating further indexing-focused experiments beyond the baseline validation considered in this study.
- Concurrent workload scenarios: Evaluation of system performance under simultaneous read and write operations, simulating realistic e-commerce traffic patterns with multiple concurrent users.
- Distributed deployments: Analysis of how replication, sharding, and distributed architectures affect performance trade-offs, particularly relevant for scaling beyond single-host limitations. Recent work by Fotache et al. [22] on OLAP performance in distributed PostgreSQL and MongoDB deployments on cloud platforms demonstrates that distributed configurations can significantly alter performance characteristics compared to single-node setups.
- Query optimization analysis: Detailed examination of query execution plans and optimization strategies employed by each database system to understand the mechanisms underlying observed performance differences.
- Alternative NoSQL systems: Extension of the comparison to include other document databases (CouchDB, Couchbase) and NewSQL systems (CockroachDB, Google Spanner) to provide a more comprehensive landscape analysis. Furthermore, as discussed

by Roy-Hubara et al. [23], investigation of polyglot persistence approaches—where different database systems are strategically combined within a single application—could reveal optimal architectures for e-commerce platforms with diverse data requirements.

- Real production workloads: Validation of findings using actual e-commerce query logs and traffic patterns from operational systems rather than synthetic benchmarks.

Finally, although MongoDB demonstrated lower performance for complex queries in this study, future research should investigate scenarios where document databases provide advantages, such as handling semi-structured product reviews with highly variable attributes, storing user-generated content with unpredictable schemas, or managing product catalogs where attribute structures evolve rapidly without requiring schema migrations.

Supplementary Materials: The datasets and supporting materials used in this study are available in the Zenodo repository at <https://doi.org/10.5281/zenodo.18084839> (accessed on 20 December 2025).

Author Contributions: Conceptualization, J.U. and V.S.; methodology, J.U. and V.S.; software, J.U. and V.S.; validation, V.S. and S.A.; formal analysis, J.U. and V.S.; investigation, J.U. and V.S.; resources, S.A.; data curation, J.U.; writing—original draft preparation, J.U. and V.S.; writing—review and editing, V.S. and S.A.; visualization, J.U. and V.S.; supervision, J.U., V.S. and S.A.; project administration, V.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets generated and analyzed during the current study, including data generation scripts, raw benchmarking results, statistical calculations are publicly available in the Zenodo repository at <https://doi.org/10.5281/zenodo.18084839>.

Acknowledgments: During the preparation of this manuscript, the authors used ChatGPT-5.2/ Google Gemini 2.5 Flash for assistance with English language editing and proofreading. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ACID	Atomicity, Consistency, Isolation, Durability
AIS	Automatic Identification System
YCSB	Yahoo! Cloud Serving Benchmark
JIT compilation	Just In Time
MVCC	Multi-Version Concurrency Control
OLAP performance	Online Analytical Processing

References

1. Taipalus, T. Database Management System Performance Comparisons: A Systematic Literature Review. *J. Syst. Softw.* **2024**, *208*, 111872. [CrossRef]
2. Khan, W.; Kumar, T.; Zhang, C.; Raj, K.; Roy, A.M.; Luo, B. SQL and NoSQL Database Software Architecture Performance Analysis and Assessments—A Systematic Literature Review. *Big Data Cogn. Comput.* **2023**, *7*, 97. [CrossRef]
3. Hassan, M. Big Data, Big Decisions Choosing the Right Database. *arXiv* **2024**, arXiv:2405.02506. [CrossRef]
4. Matcha, S.; Mishra, R.; IILM University. Database Selection and Management: Choosing the Right Database (SQL vs. NoSQL) for Your Application. *Int. J. Res. Humanit. Soc. Sci. (IJRHSS)* **2025**, *13*, 68–88. [CrossRef]
5. Cheng, Y.; Zhou, K.; Wang, J. *Performance Analysis of PostgreSQL and MongoDB Databases for Unstructured Data*; Atlantis Press: Dordrecht, The Netherlands, 2019; pp. 60–62.

6. Makris, A.; Tserpes, K.; Spiliopoulos, G.; Zissis, D.; Anagnostopoulos, D. MongoDB Vs PostgreSQL: A Comparative Study on Performance Aspects. *Geoinformatica* **2021**, *25*, 243–268. [\[CrossRef\]](#)
7. Cooper, B.F.; Silberstein, A.; Tam, E.; Ramakrishnan, R.; Sears, R. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing*; Association for Computing Machinery: New York, NY, USA, 2010; pp. 143–154.
8. Jung, M.-G.; Youn, S.-A.; Bae, J.; Choi, Y.-L. A Study on Data Input and Output Performance Comparison of MongoDB and PostgreSQL in the Big Data Environment. In *Proceedings of the 2015 8th International Conference on Database Theory and Application (DTA)*; IEEE: Jeju Island, Republic of Korea, 2015; pp. 14–17.
9. Güney, E.; Ceylan, N. Response Times Comparison of MongoDB and PostgreSQL Databases in Specific Test Scenarios. In *Electrical and Computer Engineering*; Seyman, M.N., Ed.; Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering; Springer International Publishing: Cham, Switzerland, 2022; Volume 436, pp. 178–188, ISBN 978-3-031-01983-8.
10. Costa, M.; Rodrigues, M.; Baptista, P.; Wanzeller, C.; Martins, P.; Abbasi, M. Database Encryption Performance Impact on PostgreSQL and MongoDB. In *Marketing and Smart Technologies*; Reis, J.L., López, E.P., Moutinho, L., Santos, J.P.M.D., Eds.; Smart Innovation, Systems and Technologies; Springer Nature: Singapore, 2022; Volume 279, pp. 121–127, ISBN 978-981-16-9267-3.
11. Shen, W. A Performance Comparison of NoSQL and SQL Databases for Different Scales of Ecommerce Systems. Master's Thesis, Auckland University of Technology, Auckland, New Zealand, 2022. [\[CrossRef\]](#)
12. Iqbal Thirupattur, J.; Theng Tan, A.X.; Lim, S.-L.; Wong, L.H.-E.; Yong, M.J.; Veerappampalayam Easwaramoorthy, S. Comparative Performance Analysis of SQL and NoSQL Databases for Optimizing Retail and E-Commerce Operations. In *Proceedings of the 2024 9th International Conference on Communication and Electronics Systems (ICCES)*; IEEE: Coimbatore, India, 2024; pp. 1063–1070.
13. Aboutorabi, S.H.; Rezapour, M.; Moradi, M.; Ghadiri, N. Performance Evaluation of SQL and MongoDB Databases for Big E-Commerce Data. In *Proceedings of the 2015 International Symposium on Computer Science and Software Engineering (CSSE)*; IEEE: Tabriz, Iran, 2015; pp. 1–7.
14. Abbas, N.; Farah, J. Optimizing E-Commerce Databases: A Comparative Analysis of SQL and NoSQL Solutions. Master's Thesis, Ternopil Ivan Puluj National Technical University, Ternopil, Ukraine, 2023.
15. Nakhare, D. A Comparative Study of SQL Databases and NoSQL Databases for E-Commerce. *Int. J. Res. Appl. Sci. Eng. Technol.* **2021**, *9*, 409–412. [\[CrossRef\]](#)
16. Song, I.-Y.; Whang, K.-Y.; Korea, T. Database Design for Real-World E-Commerce Systems. *IEEE Data Eng. Bull.* **2000**, *23*, 23–28.
17. Yedilkhan, D.; Mukasheva, A.; Bissengaliyeva, D.; Suyunulayev, Y. Performance Analysis of Scaling NoSQL vs SQL: A Comparative Study of MongoDB, Cassandra, and PostgreSQL. In *Proceedings of the 2023 IEEE International Conference on Smart Information Systems and Technologies (SIST)*, Astana, Kazakhstan, 4–6 May 2023; pp. 479–483.
18. Carvalho, I.; Sá, F.; Bernardino, J. Performance Evaluation of NoSQL Document Databases: Couchbase, CouchDB, and MongoDB. *Algorithms* **2023**, *16*, 78. [\[CrossRef\]](#)
19. Kuszera, E.M.; Peres, L.M.; Didonet Del Fabro, M. Exploring Data Structure Alternatives in the RDB to NoSQL Document Store Conversion Process. *Inf. Syst.* **2022**, *105*, 101941. [\[CrossRef\]](#)
20. El Khatib, M. Performance Evaluation of PostgreSQL, MongoDB and Redis for Embedded Driver Monitoring Systems: Benchmarking PostgreSQL, MongoDB and Redis for Real-Time Driver Monitoring on a Raspberry Pi. Bachelor's Thesis, University of Skövde, Skövde, Sweden, 2023.
21. Axell, C.; Schøien, E.; Thon, I.L. Insertion Speed of Indexed Spatial Data: Comparing MySQL, PostgreSQL and MongoDB. Master's thesis, Norwegian University of Science and Technology, Trondheim, Norway, 2022.
22. Fotache, M.; Badea, C.; Cluci, M.-I.; Pinzaru, C.; Eşanu, C.-S.; Rusu, O. OLAP Performance of Distributed PostgreSQL and MongoDB on OpenStack. Preliminary Results on Smaller Scale Factors. In *Proceedings of the 2024 23rd RoEduNet Conference: Networking in Education and Research (RoEduNet)*, Bucharest, Romania, 19–20 September 2024; pp. 1–6.
23. Roy-Hubara, N.; Shoval, P.; Sturm, A. Selecting Databases for Polyglot Persistence Applications. *Data Knowl. Eng.* **2022**, *137*, 101950. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.